Backend & Database Communication: How application code, ORMs, and query layers actually talk to a database.

PART 1: FOUNDATIONS

What Is a Backend?

The backend is the server-side application that receives requests, enforces business rules, and decides what is allowed to happen and what gets persisted. It does not render UI; it serves clients.

- **Receives requests:** HTTP, gRPC, message queues, WebSockets.
- **Enforces the rules:** Validation, auth, business logic.
- **Talks to the database:** Reads/writes state that outlives the request.
- **Returns a response:** Shapes results into the expected contract (JSON, HTML, etc.).

What Is a Database?

A separate, managed system (e.g., PostgreSQL, MySQL) whose job is to make state outlive the process that created it. It is a networked system with its own protocol and guarantees.

Key Concepts:

Durability: Writes survive crashes (disk vs. RAM).

Structure: Tables, rows, columns, and types.

Concurrency: Transactions and locks manage simultaneous access.

PART 2: SQL, DRIVERS & ORMS

What Is an ORM?

Object-Relational Mapping (ORM) is a bridge between object-oriented code and relational databases. It translates method calls into SQL and SQL results into objects.

Why Use an ORM?

It trades some control for productivity. It handles boilerplate CRUD, prevents SQL injection by default (via parameter binding), and allows portability across database engines.

PART 3: THE REQUEST LIFECYCLE

A typical request involves several hops:

1. Client sends request (e.g., HTTP GET).
2. Service logic validates and authenticates.
3. ORM builds a lazy query description.
4. ORM compiles query to SQL and sends via driver.
5. Database executes query.
6. Resulting rows are streamed back.
7. ORM hydrates rows into objects.
8. Response is serialized and returned.